HƯỚNG DẪN THỰC HÀNH

PHASE 2 — XÂY DỰNG HỆ THỐNG QTKD | TUẦN 05, 06, 07

Công nghệ: C# · ASP.NET Core Web API · Entity Framework Core · PostgreSQL · xUnit

1. Mục tiêu và nội dung thực hành

Ở Phase 1, sinh viên đã tìm hiểu yêu cầu và thiết kế hệ thống QTKD. Trong Phase 2, sinh viên sử dụng chính các kết quả đó để xây dựng một phần hệ thống chạy được. Không làm ba bài riêng: tuần sau tiếp tục trên mã nguồn của tuần trước.

Tuần	Công việc chính	Kết quả cần có
05	Tạo dự án, cơ sở dữ liệu và API đọc dữ liệu	Ứng dụng chạy được; có Customer, Contract và PriceList trong PostgreSQL
06	Viết các chức năng và kiểm tra quy tắc nghiệp vụ	Tạo Customer → tạo Contract → gửi trình duyệt; quản lý bảng giá cơ bản
07	Viết kiểm thử, sửa lỗi và hoàn thiện	Chạy lại được toàn bộ luồng và có bộ test tự động

Ví dụ xuyên suốt: Nhân viên Kinh doanh tạo khách hàng Công ty Minh Phát, lập hợp đồng HD-2026-018, bổ sung thông tin còn thiếu rồi gửi hợp đồng trình duyệt. Tuần 05 chuẩn bị dữ liệu; tuần 06 lập trình luồng này; tuần 07 kiểm thử chính luồng đó.

2. Chuẩn bị chung

- Mang theo kết quả Phase 1: User Story US03, Acceptance Criteria, sơ đồ trạng thái hợp đồng và các quy tắc CTR-01, CTR-02, CTR-03. Nếu thiết kế trước đây chưa phù hợp, sinh viên cập nhật và ghi lý do.
- Cài .NET SDK, PostgreSQL, Git và một IDE phù hợp. Dùng cùng phiên bản .NET trong suốt bài thực hành.
- Tạo một Git repository cá nhân. Từ tuần 05 đến tuần 07 chỉ dùng repository này; không tạo lại dự án mới ở mỗi tuần.
- Phạm vi bắt buộc: Customer, Contract, PriceList/PriceItem và thao tác gửi hợp đồng trình duyệt. Chưa yêu cầu xây dựng đầy đủ quy trình Approval, Payment, giao diện web hoặc ký điện tử.

3. Tuần 05 — Tạo nền tảng cho QTKD

3.1. Mục tiêu

Cuối tuần, ứng dụng Web API phải kết nối được PostgreSQL và đọc được dữ liệu từ database. Các bảng tạo trong tuần này sẽ được dùng tiếp ở tuần 06, không thay bằng dữ liệu viết sẵn trong code.

3.2. Làm lần lượt theo các bước sau

1. Tạo solution QTKD và một project ASP.NET Core Web API. Có thể tách thành nhiều project hoặc tổ chức theo thư mục, nhưng phải phân biệt rõ API, xử lý nghiệp vụ và truy cập dữ liệu.
2. Từ mô hình dữ liệu Phase 1, tạo các entity Customer, Contract, PriceList, PriceItem. Tối thiểu: Customer có Id, Name, Status; Contract có Id, ContractNo, CustomerId, StartDate, EndDate, Status; PriceList có Id, Name, EffectiveDate; PriceItem có PriceListId, ServiceCode, UnitPrice. Các trường này là gợi ý để thực hành, không thay thế đặc tả sinh viên đã xác nhận.

3. Tạo QtkdDbContext, khai báo quan hệ Contract–Customer và PriceList–PriceItem. Cấu hình chuỗi kết nối PostgreSQL; không đưa mật khẩu lên Git.
4. Tạo migration đầu tiên, cập nhật database và kiểm tra các bảng, khóa chính, khóa ngoại đã được tạo.
5. Thêm dữ liệu mẫu: ít nhất 2 Customer, 1 Contract ở trạng thái Draft và 1 PriceList có 2 PriceItem.
6. Viết GET /api/customers và GET /api/contracts/{id}. Gọi API bằng Swagger/Postman, sau đó đối chiếu kết quả với dữ liệu trong PostgreSQL.
7. Viết README: cách cấu hình database, chạy migration, khởi động API và thử hai endpoint. Nhờ một máy hoặc môi trường khác làm theo README để kiểm tra.

3.3. Lệnh khởi tạo tham khảo

```
dotnet new sln -n QTKD
dotnet new webapi -n QTKD.Api
dotnet sln QTKD.sln add QTKD.Api/QTKD.Api.csproj
dotnet add QTKD.Api package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add QTKD.Api package Microsoft.EntityFrameworkCore.Design
dotnet ef migrations add InitialCreate --project QTKD.Api
dotnet ef database update --project QTKD.Api
dotnet run --project QTKD.Api
```

Lưu ý: Cần cài công cụ dotnet-ef nếu máy chưa có. Nếu DbContext nằm ở project khác, điều chỉnh tham số --project và --startup-project.

3.4. Nộp cuối tuần 05

- Link repository có mã nguồn, migration và README.
- Ảnh hoặc video ngắn: chạy API, đọc Customer/Contract và xem dữ liệu tương ứng trong PostgreSQL.
- Sơ đồ hoặc hình mô tả ngắn các thành phần chính của solution.

4. Tuần 06 — Lập trình chức năng trên nền tuần 05

4.1. Mục tiêu

Tiếp tục dùng database và solution của tuần 05. Tuần này sinh viên viết API để tạo, sửa và xử lý hợp đồng. Không chỉ làm CRUD: các quy tắc nghiệp vụ phải được kiểm tra trước khi lưu.

4.2. Các chức năng cần làm

Phần	API/hành vi tối thiểu	Điều cần kiểm tra
Customer	Tạo; xem danh sách/chi tiết; cập nhật	Tên và các trường bắt buộc; Customer được tham chiếu phải hợp lệ
Contract	Tạo; xem; cập nhật; gửi trình duyệt	Ngày hiệu lực; trạng thái; CTR-01, CTR-02, CTR-03
Pricing	Tạo và xem PriceList; thêm và xem PriceItem	Đơn giá hợp lệ; liên kết PriceItem–PriceList đúng

4.3. Làm kỹ US03 — Gửi hợp đồng trình duyệt

8. Mở lại US03 và các Acceptance Criteria của Phase 1. Ghi rõ điều kiện gửi hợp đồng, trạng thái trước/sau và trường hợp phải từ chối.
9. Tạo endpoint đề xuất POST /api/contracts/{id}/submit. Thống nhất mẫu phản hồi thành công và phản hồi lỗi; ghi vào tài liệu API.
10. Trong service, tìm Contract theo id, kiểm tra trạng thái Draft, Customer hợp lệ, ngày hiệu lực và nội dung/tài liệu bắt buộc theo CTR-02. Chỉ khi hợp lệ mới đổi sang PendingApproval và lưu.

11. Khi thiếu thông tin hoặc trạng thái không phù hợp, API trả lỗi rõ ràng và không thay đổi Contract. Không cho chuyển thẳng Draft sang Approved.
12. Thử gửi cùng một hợp đồng hai lần. Lần sau không được tạo thêm một lần xử lý tương đương. Vì chưa xây dựng Approval đầy đủ, sinh viên cần ghi rõ cách mô phỏng việc gửi và giới hạn của phiên bản hiện tại.
13. Thực hiện demo theo thứ tự: tạo Customer → tạo Contract Draft → gửi khi thiếu dữ liệu (thất bại) → bổ sung dữ liệu → gửi lại (thành công) → xem trạng thái trong database.

4.4. Các quy tắc cần đối chiếu

Mã	Ý nghĩa	Cách kiểm tra
CTR-01	Chỉ cho sửa hợp đồng ở Draft hoặc Revision Requested	Thử sửa Contract ở PendingApproval: phải bị từ chối
CTR-02	Chỉ gửi khi Customer, ngày hiệu lực, nội dung/tệp bắt buộc hợp lệ	Thiếu một điều kiện: gửi thất bại, dữ liệu không đổi
CTR-03	Không được chuyển Draft thẳng sang Approved	Submit thành công chỉ chuyển sang PendingApproval

Nếu Phase 1 chưa xác định danh sách tệp/nội dung bắt buộc, sinh viên phải ghi rõ giả định dùng trong bản demo và phần cần xác nhận; không tự xem đó là yêu cầu chính thức.

4.5. Nộp cuối tuần 06

- Cập nhật mã nguồn vào repository của tuần 05; migration bổ sung nếu có.
- Danh sách API kèm request/response mẫu và bảng đối chiếu US03/CTR-01–03 với đoạn xử lý tương ứng.
- Ảnh hoặc video demo toàn bộ luồng Customer → Contract → Submit, có cả trường hợp thành công và thất bại.

5. Tuần 07 — Kiểm thử chính phần đã làm ở tuần 06

5.1. Mục tiêu

Không xây thêm một hệ thống khác. Sinh viên dùng mã nguồn tuần 06, viết test để kiểm tra các chức năng đã có, sửa lỗi phát hiện được và chuẩn bị phiên bản chạy ổn định.

5.2. Các bước thực hiện

14. Tạo project xUnit riêng, tham chiếu phần code cần kiểm thử. Tách test nghiệp vụ (unit test) và test gọi API/database (integration test).
15. Viết unit test cho CTR-01, CTR-02 và CTR-03. Mỗi rule có ít nhất một trường hợp đúng và một trường hợp sai.
16. Viết integration test cho các API chính: tạo Customer, tạo Contract, gửi Contract và đọc lại trạng thái. Dùng database kiểm thử riêng, không dùng database có dữ liệu quan trọng.
17. Kiểm tra cả HTTP status, nội dung trả về và dữ liệu sau khi gọi API. Không kết luận thành công chỉ vì API trả 200.
18. Chạy lại luồng demo tuần 06 sau khi sửa lỗi. Cập nhật README và ghi lại lỗi còn tồn tại, nếu có.

5.3. Danh sách test tối thiểu

Test	Dữ liệu/thao tác	Kết quả mong đợi
TC01	Tạo Customer hợp lệ	Tạo được và đọc lại từ DB
TC02	Tạo Customer thiếu thông tin bắt buộc	Bị từ chối; có thông báo lỗi
TC03	Sửa Contract ở Draft	Thành công khi dữ liệu hợp lệ

TC04	Sửa Contract ở PendingApproval	Bị từ chối; dữ liệu giữ nguyên
TC05	Submit Contract thiếu điều kiện CTR-02	Bị từ chối; vẫn là Draft
TC06	Submit Contract hợp lệ	Chuyển sang PendingApproval
TC07	Thử chuyển Draft thẳng Approved	Bị từ chối
TC08	Submit lặp một Contract	Không tạo thêm kết quả xử lý trùng

5.4. Cách nộp minh chứng kiểm thử

- Chạy dotnet test và lưu kết quả (ảnh console hoặc file TRX).
- Nộp bảng TC01–TC08: Pass/Fail, test nào tự động, test nào chạy thủ công và lý do nếu chưa tự động hóa.
- Ghi ít nhất một lỗi đã tìm thấy (nếu có): cách tái hiện, nguyên nhân và commit sửa lỗi. Nếu không phát hiện lỗi, ghi rõ đã thử những tình huống nào.
- Demo cuối: clone → cấu hình → migration → chạy API → chạy test.

6. Quy định nộp bài chung

Giữ một repository xuyên suốt cả ba tuần. Đề nghị bố trí: src/ (mã nguồn), tests/ (kiểm thử), docs/ (thiết kế/API/minh chứng), README.md. Thời hạn và nơi nộp theo thông báo trên Moodle.

Sinh viên có thể dùng AI Coding Assistant để tìm lỗi, tham khảo cách tổ chức code hoặc gợi ý test. Tuy nhiên, sinh viên phải giải thích được mã nguồn đã thực hiện và tự chạy kiểm chứng. Không nộp mã nguồn chưa đọc hoặc chưa thử.